Lottery Scheduler and Default Scheduler Comparison

by

Pınar İzmir

Ece Düzgeç

Bihter Havuçcuoğlu

CSE 331 Operating Systems Design
Term Project Report

Yeditepe University
Faculty of Engineering
Department of Computer Engineering
Fall 2025

ABSTRACT

CPU scheduling plays an important role in how an operating system shares processor time among running processes. In this project, an alternative scheduling approach called Lottery Scheduling is implemented in the Linux kernel and compared with the default Linux scheduler in order to compare both scheduling approaches. In Lottery Scheduling, each process is assigned a number of tickets, and the scheduler randomly selects a ticket to decide which process will run next. Processes with more tickets have a higher chance of being scheduled, which allows priority and fairness to be handled in a flexible way.

The Linux scheduler is modified so that all processes are managed using the Lottery Scheduling algorithm. In addition, a new system call is added to enable or disable the lottery scheduler during runtime. To evaluate the behavior of the new scheduler, performance measurements are collected using CPU-intensive processes. Metrics such as CPU time distribution are analyzed and compared with the original scheduler. The results show how Lottery Scheduling affects fairness and CPU usage, and they help demonstrate the impact of different scheduling policies on system performance.

Contents

1 INTRODUCTION **3**

2 DESIGN and IMPLEMENTATION **4**
2.1 OPERATION IN THE LINUX SCHEDULER 4
2.2 DESIGN OF THE LOTTERY SCHEDULER 4
2.2.1 DYNAMIC TICKET ADJUSTMENT 5
2.2.2 LOTTERY SELECTION PROCESS 6
2.2.3 KERNEL MODIFICATIONS 6
2.2.4 SWITCHING SCHEDULER VIA SYSTEM CALL 9
2.2.5 PROCESS INITIALIZATION 9

3 TESTS and RESULTS **10**
3.1 EXPERIMENTAL SETUP 10
3.2 TEST RESULTS 12
3.2.1 Test 1 Results(4 Processes): 12
3.2.2 Test 2 Results (5 Processes): 13
3.2.3 Test 3 Results(10 Processes): 14
3.2.4 Overall Test Comparative Analysis 15

4 CONCLUSION **16**

5 REFERENCES **17**

1 INTRODUCTION

Process scheduling is one of the most important responsibilities of an operating system. Since multiple processes may be ready to run at the same time, the scheduler decides which process will use the CPU and for how long. The choice of scheduling algorithm directly affects system performance, **fairness** among processes, and overall responsiveness.

The Linux Operating System uses a complex scheduling mechanism designed to balance **efficiency** and **fairness** for general-purpose workloads. We discussed if there is any possible scheduler that is better than Linux's **Default Scheduler** in terms of **CPU usage** and through this project; we wanted to demonstrate how the **Lottery Scheduler** works, measure its performance, and compare it with the **Default Linux Scheduler**.

The design, implementation, and evaluation of the Lottery Scheduler to Default Scheduler are developed and presented in this report as part of the Phase 2 of our term project. In order to do that we created a system call to enable dynamic switching between the default scheduler and the Lottery Scheduler at runtime.

Our report includes the project structure, the implementation details of the Lottery Scheduler, the tests we performed, the results we obtained, and our conclusions from them. The entire project and tests were carried out on a laptop with an Intel i7 processor and Windows operating system, using VirtualBox to create a virtual machine called CSE331 containing the kernel 2.4.27 source code. Further information about tests will be explained in the **TESTS and RESULTS** section.

To test the performance of the scheduler, we used **Linux commands, awk** and **shell** scripts. We tested with varying numbers of processes, and multiple test runs to collect trustworthy results. Through these test runs, we observed and analyzed the **behavior** and **fairness** of the Lottery Scheduling algorithm compared to Linux Default Scheduler.

2 DESIGN and IMPLEMENTATION

This section explains the operation of the default Linux scheduler, describes the design and behavior of the implemented Lottery Scheduler, compares the two scheduling approaches, and presents the methodology used in this project. Key parts of the implementation are explained using pseudocode rather than full source code to improve clarity and readability.

2.1 OPERATION IN THE LINUX SCHEDULER

The Linux scheduler is responsible for selecting one runnable process to execute whenever the CPU becomes available. In Linux kernel version 2.4.27, scheduling decisions are primarily based on process states, priorities, and time slices.

Each process is represented by a `task_struct`, which contains scheduling-related fields such as **the process state**, **priority**, and **remaining time quantum**. Only processes in the TASK_RUNNING state are considered eligible for execution.

The scheduler is invoked after system calls, timer interrupts or when a running process voluntarily yields or blocks. During scheduling, Linux iterates over all runnable processes and selects the next process based on the active scheduling policy (SCHED_OTHER, SCHED_FIFO or SCHED_RR). For the default SCHED_OTHER policy, a dynamic priority is calculated using the process's **nice** value that when this value for the process increases, the scheduler adjusts to take that process later, meaning it behaves more "politely", and its recent **CPU usage**. Processes with lower nice values are favored, while time slicing ensures that no process monopolizes the CPU.

In simplified form, the Default Scheduler operates as in **Algorithm 1**.

Algorithm 1 Default Scheduler

```
for each runnable process do
    calculate dynamic priority
end for
select process with highest priority
dispatch selected process to CPU
```

This approach is deterministic and priority driven, which can lead to unfair CPU distribution under heavy CPU bound workloads.

2.2 DESIGN OF THE LOTTERY SCHEDULER

The Lottery Scheduler is a **probabilistic** scheduling algorithm designed to provide **proportional share** CPU scheduling, so we designed it to select the next runnable process based on this approach. Each process is assigned a certain number of tickets, and the probability of a process being scheduled is corresponding to the number of tickets it owns. Unlike traditional priority-based schedulers, this approach provides a simple and flexible way to achieve proportional-share scheduling. In this way, a runnable process is assigned a number of tickets, which represents its **chance** of being scheduled.

In this project, when the Lottery Scheduler is enabled, **all processes are scheduled exclusively using the lottery mechanism**, regardless of their original scheduling policy. This simplifies the design and ensures consistency during experimentation.

In order to simplify the implementation, the Lottery Scheduler is designed to manage all runnable processes when enabled, regardless of their original scheduling policy. This design decision avoids complex interactions with existing policies such as SCHED_FIFO and SCHED_RR, and ensures that scheduling behavior is fully controlled by the lottery mechanism. The Lottery Scheduler implementation consists of **three** main components that work together to provide fair probabilistic scheduling:

2.2.1 DYNAMIC TICKET ADJUSTMENT

The $\mathbf{update_tickets_based_on_time}()$ function implements an adaptive ticket allocation mechanism that responds to process behavior. Before each scheduling decision, the function examines each runnable process and adjusts its ticket count based on the elapsed time since its last CPU allocation.

- **Lose Ticket Time (C) (<20ms):** The process loses one ticket (minimum 1). This mechanism penalizes CPU-intensive processes that recently consumed CPU time, reducing their probability of immediate rescheduling.
- **Gain Ticket Time (D) (>200ms):** The process gains one ticket (maximum 5). This rewards the waiting processes by increasing their scheduling probability, preventing starvation of interactive processes.
- **Unchanged Ticket Time (C-D) (20-200ms):** Ticket count remains unchanged, creating a neutral zone that avoids excessive ticket fluctuation.

Additionally, time calculations use the kernel's jiffies counter, converting to milliseconds using the formula:

$$\text{time_ms} = \frac{\text{jiffies_diff} \times 1000}{Hz}$$

This ensures consistency across different kernel configurations with varying Hz values (typically 100, 250, or 1000 Hz).

Each process maintains a ticket count within given limits. The number of tickets is dynamically adjusted based on the **process's recent CPU usage**. Processes that **have not used the CPU for a long time gain tickets**, while processes that are scheduled **frequently lose tickets**. This mechanism introduces a form of aging and **helps prevent starvation**.

2.2.2 LOTTERY SELECTION PROCESS

The $\mathbf{select_by_lottery}()$ function implements the core lottery mechanism through a **three-step process**:

Step 1: The function iterates through all processes in the TASK_RUNNING state, updating their tickets and calculating the total ticket pool. This sum represents the total number of chances available in the current lottery draw.

Step 2: A pseudo-random number is generated in the range [0, total_tickets-1] using a linear generator based on jiffies

$$\text{winning_ticket} = (\text{jiffies} \times 1103515245 + 12345) \,\%\, \text{total_tickets}$$

This generator provides randomness for scheduling purposes with minimal overhead.

Step 3: The function iterates through runnable processes, accumulating ticket counts until the total exceeds the winning ticket number. The process at this point wins the lottery and is selected for execution. Upon selection, the winner's last_cpu_time is updated to the current jiffies value.

2.2.3 KERNEL MODIFICATIONS

We firstly made modifications in the **usr/src/kernel-source-2.4.27/include/linux/sched.h** file where declaring A, B, C, D, E values given us from our project constraints. Our implementation adheres to the B group parameters: A=5 (MAX_TICKETS), B=1 (MIN_TICKETS), C=20ms (LOSE_TICKET_TIME), D=200ms (GAIN_TICKET_TIME), and E=3 (INITIAL_TICKET). This configuration creates a dynamic scheduling environment where ticket allocation adapts based on recent CPU usage.

Three new fields were added to track lottery-specific information for each process to the task_struct to store lottery-related information, such as the current ticket count and the timestamp of the last CPU execution to support lottery scheduling.

- **lottery_tickets** (unsigned int): Maintains the current number of tickets held by the process. This value dynamically adjusts based on CPU usage patterns and is bounded by MIN_TICKETS (1) and MAX_TICKETS (5).
- **last_cpu_time** (unsigned long):Records the jiffies timestamp when the process last received CPU time. This timestamp is critical for calculating time-based ticket adjustments.
- **original_tickets** (unsigned int): Stores the initial ticket allocation (E=3) for reference purposes and potential reset operations.

These fields were strategically positioned in the task_struct to avoid disrupting the existing cache-line optimization for the schedulerś $\mathbf{goodness}()$ loop. The placement ensures minimal performance impact on the Default Scheduler when the lottery scheduler is disabled.

Then we continued with **usr/src/kernel-source-2.4.27/kernel/sched.c** file and declared additional macros named $\mathbf{LOTTERY_ENABLED}$ and $\mathbf{DEFAULT_SCHEDULER}$; also additional functions such as update_tickets_based_on_time(), get_total_tickets(), *select_by_lottery().

I. **update_tickets_based_on_time**() : Adjusts process scheduling priority dynamically using a lottery ticket system. Processes that recently used the CPU are penalized (lose tickets), while processes that have been waiting longer are rewarded (gain tickets), ensuring fair CPU time distribution.

Algorithm 2 update_tickets_based_on_time

```
function UPDATE_TICKETS_BASED_ON_TIME(p : Process)
    current_time ← get_jiffies()
    if p.last_cpu_time == 0 then
        return
    end if
    elapsed_ms ← ((current_time − p.last_cpu_time) × 1000)/HZ
    if elapsed_ms < LOSE_TICKET_TIME then
        p.lottery_tickets ← max(MIN_TICKETS, p.lottery_tickets − 1)
    else if elapsed_ms > GAIN_TICKET_TIME then
        p.lottery_tickets ← min(MAX_TICKETS, p.lottery_tickets + 1)
    end if
end function
```

II. **get_total_tickets**() : This function calculates the total number of lottery tickets across all currently runnable processes. Before summing tickets, it updates each process's ticket count based on how long they've been waiting, ensuring recently active processes have lower priority. This total is used for the lottery scheduler's random selection algorithm.

Algorithm 3 get_total_tickets

```
function GET_TOTAL_TICKETS
    total_tickets ← 0
    for each process p in system do
        if p.state == TASK_RUNNING then
            UPDATE_TICKETS_BASED_ON_TIME(p)
            total_tickets ← total_tickets + p.lottery_tickets
        end if
    end for
    return total_tickets
end function
```

III. **select_by_lottery**() : Implements a lottery scheduler that randomly selects a runnable process based on ticket distribution. Each process's ticket count represents its scheduling probability. A winning ticket is drawn randomly, and the process whose cumulative tickets exceed the winning number is selected to run next. This ensures probabilistic fairness weighted by ticket allocation.

Algorithm 4 select_by_lottery

```
function SELECT_BY_LOTTERY
    total_tickets ← GET_TOTAL_TICKETS
    if total_tickets ≤ 0 then
        return NULL
    end if
    winning_ticket ← RANDOM_INT(0, total_tickets − 1)
    current_ticket ← 0
    for each process p in system do
        if p.state == TASK_RUNNING then
            current_ticket ← current_ticket + p.lottery_tickets
            if current_ticket > winning_ticket then
                p.last_cpu_time ← CURRENT_TIME
                return p
            end if
        end if
    end for
    return NULL
end function
```

We added an if-else condition inside the **schedule()** function which is a scheduler dispatcher that switches between Lottery Scheduling and the Default Scheduler. If Lottery Scheduling is enabled, the scheduler first iterates over all runnable tasks (TASK_RUNNING) and computes the total number of tickets. A random number is then generated within this range, and the corresponding process is selected by cumulatively summing ticket values until the winning ticket is reached. If no process is found (no tickets available), it falls back to the Default Scheduler's selection logic. Otherwise, it continues with the lottery-selected process.

Algorithm 5 repeat_schedule

```
if current_scheduler == LOTTERY_ENABLED then
    next ← SELECT_BY_LOTTERY
    if next == NULL then
        next, counter ← DEFAULT_SCHEDULER_SELECT
    else
        counter ← 1
    end if
else
    next, counter ← DEFAULT_SCHEDULER_SELECT
end if
```

2.2.4 SWITCHING SCHEDULER VIA SYSTEM CALL

A new system call named **sys_set_scheduler** with system call number 260 is implemented to allow the Lottery Scheduler to be enabled or disabled **dynamically at runtime**. A global variable **current_scheduler** controls which scheduling algorithm is active, allowing runtime switching without kernel recompilation. When current_scheduler is set to LOTTERY_ENABLED (1), the **select_by_lottery**() function is invoked. If it successfully selects a process, execution proceeds directly to context switching. If selection fails (returning NULL), the system falls back to the default scheduler, ensuring system stability even in edge cases.

Algorithm 6 sys_set_scheduler

```
function SET_SCHEDULER(mode: Integer)
    if mode == LOTTERY_ENABLED then
        current_scheduler ← LOTTERY_ENABLED
        "Lottery scheduler enabled"
    else if mode == DEFAULT_SCHEDULER then
        current_scheduler ← DEFAULT_SCHEDULER
        "Default scheduler enabled"
    else
        return ERROR_INVALID_ARGUMENT
    end if
    return SUCCESS
end function
```

The integration point occurs immediately after process state handling and before the Default Scheduler's **goodness**() loop. Integration with the system call table required modifications to three files:

- **sr/src/kernel-source-2.4.27/arch/i386/kernel/entry.S:** Added the system call entry point
- **usr/src/kernel-source-2.4.27/include/asm-i386/unistd.h:** Defined __NR_set_scheduler as 260
- **usr/src/kernel-source-2.4.27/kernel/sched.c:** Implemented the sys_set_scheduler() function

2.2.5 PROCESS INITIALIZATION

Proper initialization of lottery-specific fields is critical for correct scheduler operation. In **usr/src/kernel-source-2.4.27/kernel/fork.c**, the **do_fork**() function was modified to initialize the three lottery fields for each newly created process:

- **p->lottery_tickets = INITIAL_TICKETS:** The lottery_tickets field is set to INITIAL_TICKETS (3), giving all new processes equal starting priority.
- **p->original_tickets = INITIAL_TICKETS:** The original_tickets field stores this initial value for potential future use.
- **p->last_cpu_time = jiffies:** Establishing a baseline for time-based ticket adjustments, creating a basis for time based ticketing system.

This initialization occurs after the new process's basic fields are copied from the parent but before the process is added to the run queue, ensuring that lottery fields are properly set before the scheduler might examine them.

3 TESTS and RESULTS

To test the performance of the scheduler, we used Linux command-line tools, awk and shell scripts. Performance tests were conducted using CPU-intensive workloads and varying numbers of processes. Multiple test runs were performed to collect trustworthy and consistent results. Through these tests, we observed how CPU time was distributed among processes and analyzed the fairness and behavior of the Lottery Scheduling algorithm.

To evaluate the behavior of the scheduler, three non-root users are created on the system. For each user, a simple CPU-intensive program containing an infinite loop is placed in each user's home directory. These programs are executed simultaneously to generate continuous CPU bursts. All programs are started at approximately the same time to ensure fair competition for CPU resources. The workload is designed to be purely CPU-bound, eliminating the effects of I/O or blocking operations on scheduling behavior.

3.1 EXPERIMENTAL SETUP

To compare the performance of our newly created Lottery Scheduler and the existing Default Scheduler, we created a total of three test cases. These test cases involved varying the number of processes performing the same operation within the created users (ece, pinar, bihte) and recording the CPU usage percentages in separate text files. The Linux commands used for this process are as follows:

- **nano infinite.c :** To simulate continuous process execution, we created a file named **infinite.c** containing an infinite loop.
- **gcc infinite.c -o infinite:** With this Linux command, infinite.c was subsequently compiled within a Linux environment to generate an executable named infinite.
- **./infinite & :** This is used to execute the infinite in the background, an "&" is placed immediately after the termination command. This command is written as many times as the number of processes to be created for each user.
- **-top -n 100 -d 1 -b > SchedulerNtestTESTCASE.txt:** This Linux command, using the "top" command, plots a detailed table of all processes running within the kernel environment. It performs this process 100 times, with a delay of one second each time, and writes each resulting table to a text file named SchedulerNtestTESTCASE.txt. The file is named "default" if the output is generated using the Default Scheduler, and "lottery" if the output is generated using the Lottery Scheduler. The number "N" indicates how many times the 100-times print operation is performed (Maximum value is 10), and "TESTCASE" specifies which test case it belongs to.
- **MSE_calculator.sh:** This shell script automates the calculation of Mean Square Error (MSE) for scheduler performance testing by processing CPU utilization data from our test files. It extracts per-process CPU usage percentages, computes the ideal fair share (100% divided by the number of processes), and calculates MSE by averaging the squared differences between observed and ideal CPU allocation for each Process ID (PID). The script produces two key outputs for both the Default and our Lottery Scheduler: a detailed table showing the MSE for each individual process (PID) and the average of all per-PID MSE values. This MSE is the single, reinforced fairness metric that we used to create comparative tables, as a lower MSE indicates a fairer scheduler.

Test 1: 4 concurrent CPU-intensive processes that 2 processes from user bihte, 1 process from user ece and lastly 1 process from user pinar are created. This scenario simulates a lightly loaded system with minimal contention.

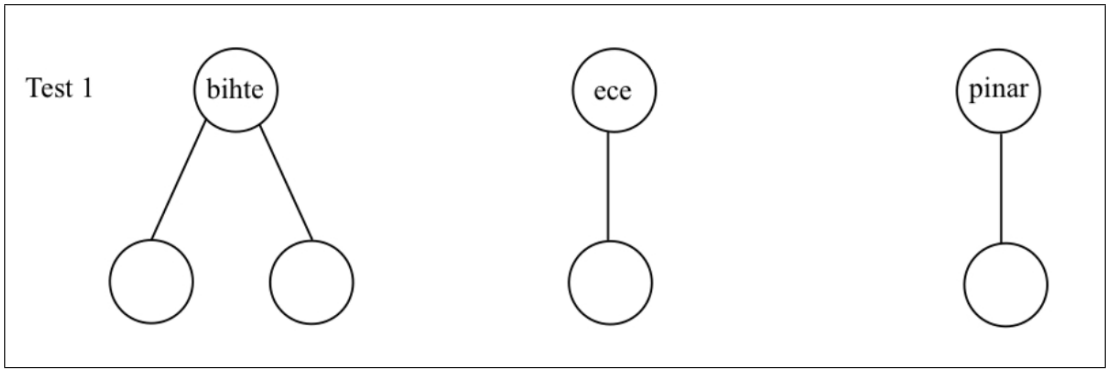

Figure 1: The schema of the processes created for Test 1 from each user.

Test 2: 5 concurrent processes that 1 process from user bihte, 2 processes from user ece and lastly 2 processes from user pinar are created, providing moderate CPU contention and more complex scheduling decisions.

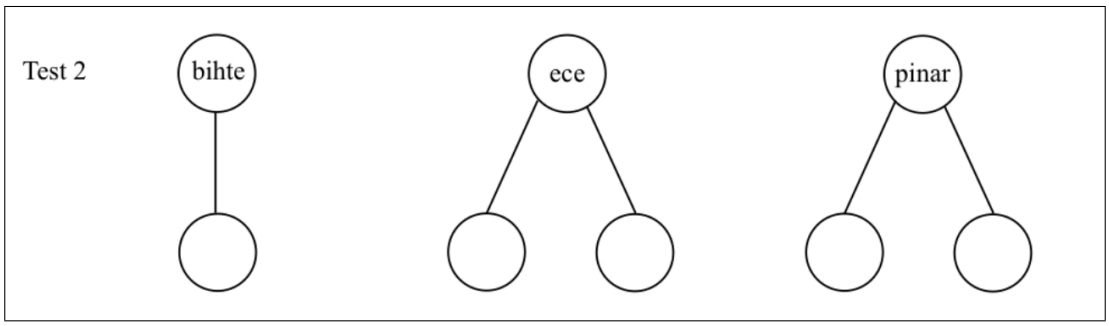

Figure 2: The schema of the processes created for Test 2 from each user.

Test 3: 10 concurrent processes that 3 processes from user bihte, 4 processes from user ece and lastly 3 processes from user pinar are created creating high contention and checking the scheduler's fairness mechanisms.

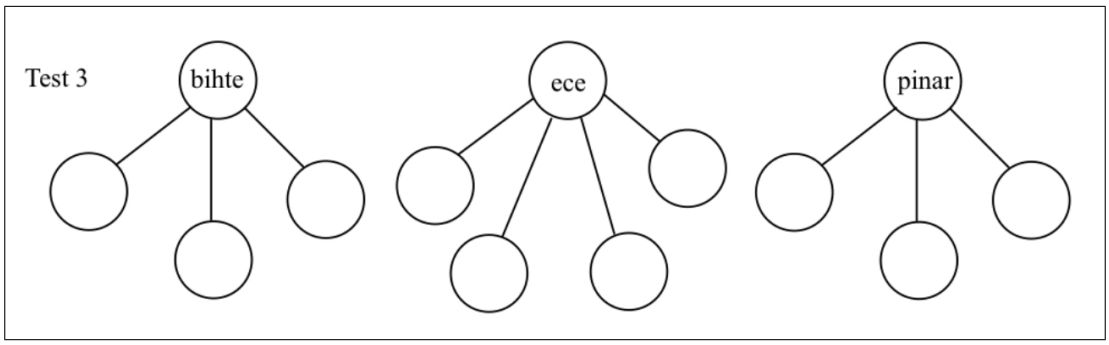

Figure 3: The schema of the processes created for Test 3 from each user.

For each test scenario, the following protocol was executed:

- System baseline established with Default Scheduler active and minimal background load.
- Test processes launched simultaneously under Default Scheduler, executing at least 60-120 seconds.
- CPU time distribution data collected using top Linux Command measurements.
- All test processes terminated and system allowed to stabilize.
- Lottery Scheduler activated via test_scheduler utility (syscall 260 with mode=1).
- Identical test workload reexecuted under Lottery Scheduler with same measurement methodology.
- Results are combined and statistical analysis performed

3.2 TEST RESULTS

Performance evaluation focused on only one primary metric that characterize scheduler behavior and fairness:

Mean Squared Error (MSE): Quantifies the variance in CPU allocation over time. Lower MSE indicates more consistent, predictable allocation patterns. Higher MSE suggests more variability but not necessarily unfairness.

3.2.1 Test 1 Results(4 Processes):

Test 1 examined 4 CPU-intensive processes from three different users (bihte, ece, pinar). Both schedulers achieved nearly perfect fairness in average CPU allocation.

Table 1: Comparison of MSE Values of each Scheduler

(a) Default Scheduler

PID	USER	MSE
756	bihte	8.9425
757	bihte	9.9152
763	ece	19.4669
769	pinar	20.0926

(b) Lottery Scheduler

PID	USER	MSE
620	bihte	27.6523
621	bihte	21.037
627	ece	23.3694
633	pinar	22.8037

MSE Analysis: While average CPU distribution was nearly identical, the variance patterns differed significantly between schedulers:

Default Scheduler MSE ranged from 8.94 to 20.09 with average of 14.61, while Lottery Scheduler MSE ranged from 21.04 to 27.65 with average of 23.72. The Lottery Scheduler exhibited higher average MSE, indicating more variability in CPU time allocation over short time periods.

Scheduler	Average	Minimum	Maximum
Default	14.6043	8.9425	20.0926
Lottery	23.7156	21.037	27.6523

Table 2: Average, minimum, maximum MSE values of both schedulers.

This increased variance is an expected characteristic of probabilistic scheduling. The lottery mechanism introduces randomness in scheduling decisions, which manifests as higher short-term variability while maintaining long-term fairness. The Default Scheduler's approach produces more predictable allocations, reflected in lower MSE values.

3.2.2 Test 2 Results (5 Processes):

Test 2 increased concurrency to 5 processes, intensifying scheduling contention and providing a more challenging fairness scenario.

Table 3: Comparison of MSE Values of Each Scheduler

(a) Default Scheduler

PID	USER	MSE
752	pinar	21.0574
753	pinar	16.1795
759	bihte	17.0612
765	ece	24.3589
766	ece	28.4107

(b) Lottery Scheduler

PID	USER	MSE
624	pinar	22.1371
625	pinar	19.7115
631	bihte	31.9206
637	ece	20.9166
638	ece	19.764

Test 2 MSE results showed interesting patterns. Default Scheduler average MSE was 21.41, while Lottery Scheduler averaged at 22.89 which is higher. This represents significantly less variance difference than Test 1, suggesting that increased contention may reduce the relative variability of probabilistic scheduling.

Scheduler	Average	Minimum	Maximum
Default	21.4135	16.1795	28.4107
Lottery	22.89	19.7115	31.9206

Table 4: Average, minimum, maximum MSE values of both schedulers.

Due to the dynamic adjustment mechanism, one lottery process which has PID 631 exhibited higher MSE with 31.92, showing a temporary reduced ticket allocation. This demonstrates the adaptiveness of the Lottery Scheduler processes that recently consumed significant CPU time may experience temporarily reduced scheduling probability.

3.2.3 Test 3 Results(10 Processes):

Test 3 represents the most demanding scenario with 10 concurrent CPU-intensive processes, creating maximum scheduling contention and providing the definitive fairness evaluation.

Table 5: Comparison of MSE Values of Each Scheduler

(a) Default Scheduler

PID	USER	MSE
749	ece	17.5765
750	ece	23.0768
751	ece	14.1226
752	ece	19.3888
758	pinar	16.1674
759	pinar	14.3304
760	pinar	20.8222
766	bihte	24.1382
767	bihte	21.9936
768	bihte	42.8702

(b) Lottery Scheduler

PID	USER	MSE
620	ece	3.7724
621	ece	4.6553
622	ece	3.75
623	ece	4.1082
629	pinar	4.3636
630	pinar	4.0401
631	pinar	3.9779
637	bihte	3.9229
638	bihte	4.2329
639	bihte	4.0359

MSE Analysis: Test 3 revealed a dramatic and unexpected result under heavy load, the lottery scheduler exhibited considerably lower MSE than the Default Scheduler with a significant reduction in average MSE of 4.08 and 21.45. This represents a complete reversal from Tests 1 and Test 2.

Scheduler	Average	Minimum	Maximum
Default	21.4487	14.1226	42.8702
Lottery	4.0859	3.75	4.6553

Table 6: Average, minimum, maximum MSE values of both schedulers.

This finding suggests that at high contention levels, the probabilistic lottery mechanism may actually produce more consistent short term CPU allocation than the Default Scheduler's deterministic approach. Several factors may contribute to this behavior:

- With 10 competing processes, the lottery's random selection may distribute scheduling opportunities more evenly across short time periods.
- The Default Scheduler's $\mathbf{goodness}()$ function may create subtle biases at high conflict that amplify short term variance
- Dynamic ticket adjustment may stabilize allocation patterns as the system reaches a balance.
- Statistical averaging effects become stronger with more competing processes

The Lottery Scheduler's MSE values clustered tightly around 3.75-4.66, indicating a highly consistency across all processes. The Default Scheduler showed much wider variance around 14.12-42.87, with one process which has PID 768 experiencing particularly high variability with MSE value 42.87.

3.2.4 Overall Test Comparative Analysis

Scheduler	Average	Minimum	Maximum
Default	19.9985	8.9425	42.8702
Lottery	13.1669	3.75	31.9206

Table 7: Overall average, minimum, maximum MSE values of both schedulers.

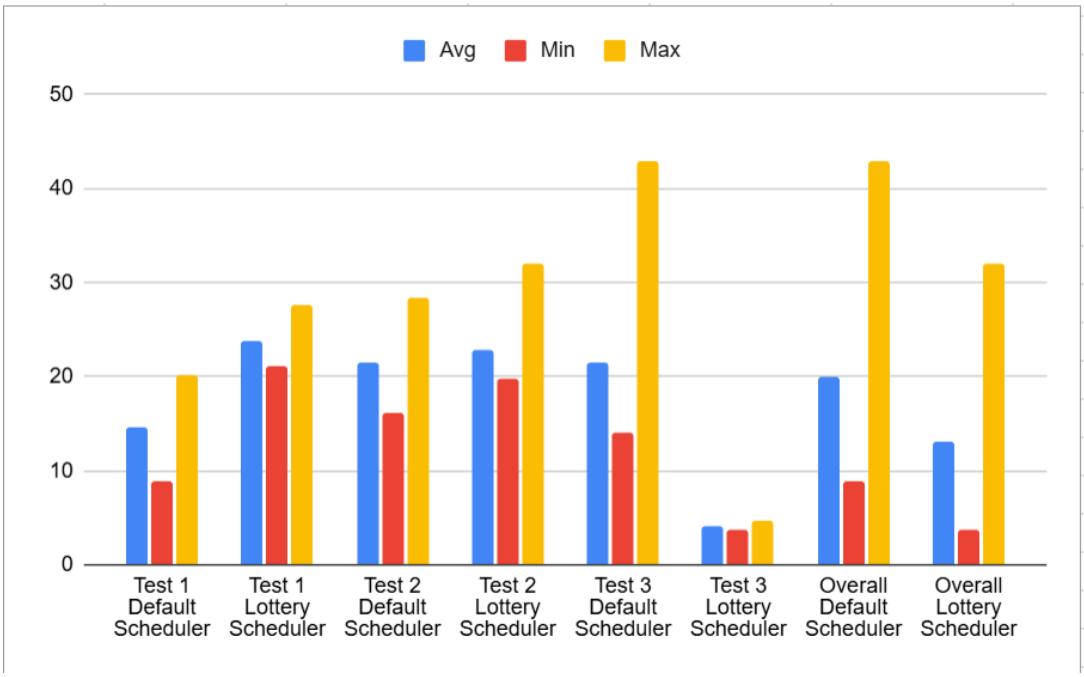

Figure 4: Overall average, minimum and maximum MSE values of both schedulers in bar chart.

Combining results across all three test scenarios reveals important patterns in scheduler behavior:

* **Fairness:** Both schedulers maintained near-perfect average CPU allocation fairness across all load conditions. Maximum deviation never exceeded from ideal allocation, demonstrating solid fairness mechanisms in both designs. The Lottery Scheduler's probabilistic approach achieves long term fairness equivalent to deterministic time slicing.
* **Variance:** MSE patterns showed strong load dependence. At 4 processes, Lottery Scheduler MSE averaged higher than default. At 5 processes, the difference narrowed. At 10 processes, Lottery Scheduler MSE dropped below default. This suggests a critical point where probabilistic scheduling becomes advantageous for consistency.
* **Scalability:** The Lottery Scheduler's better MSE performance with more processes suggests that it may scale better to highly concurrent environments. As process count increases, random selection provides more uniform distribution than deterministic approach
* **Predictability:** At low to medium conflict, the Default Scheduler offers more predictable short term behavior at lower MSE, which benefits latency applications. However, this advantage disappears with more processes, where the Lottery Scheduler's randomization actually improves consistency.

4 CONCLUSION

This project successfully implemented and evaluated a Lottery Scheduling mechanism in the Linux 2.4.27 kernel, achieving all specified design and implementation objectives. The scheduler was fully functional and supported correct lottery selection, dynamic ticket adjustment, runtime switching between schedulers, and systematic comparison with the Default Scheduler under multiple scenarios. Through both implementation and experimentation, the project demonstrated that probabilistic scheduling can be practically and effectively integrated into a real operating system kernel.

Experimental results confirmed that Lottery Scheduling provides proportional fairness in CPU allocation based on assigned ticket counts. Across all test scenarios, the deviation from ideal CPU share was minimal.

The most significant and unexpected finding of this study emerged with more processes. When the system was tested with 10 concurrent processes, the Lottery Scheduler demonstrated substantially higher consistency than the Default Scheduler, achieving significantly lower mean squared error (MSE). This result challenges the common assumption that deterministic schedulers always provide more predictable behavior. While the Default Scheduler performed slightly better with less processes, probabilistic scheduling showed better consistency as conflict increased. It suggests that statistical averaging effects in lottery selection may reduce short term allocation variance with more processes.

The implementation demonstrated that Lottery Scheduling can be achieved with minimal system overhead. Three additional fields were added to the task structure. Despite this simplicity, the scheduler delivered adaptive behavior, proportional fairness, and runtime flexibility.

Overall, this project shows that Lottery Scheduling is a feasible and superior alternative to default scheduling algorithms. While it may not be optimal for all scenarios, it offers clear advantages in proportional resource management, automatic prioritization of processes, and improved consistency under high simultaneousness. These characteristics make it especially preferable where fairness and scalability are critical.

5 REFERENCES

References

. Super User Contributors. (2012). How to find files containing two strings together in linux [Accessed: 2025-05-22]. https://superuser.com/questions/437384/how-to-find-files-containing-two-strings-together-in-linux

. Stack Overflow Contributors. (2013). Search with grep for two expressions at once [Accessed: 2025-05-22]. https://stackoverflow.com/questions/15116460/search-with-grep-for-two-expressions-at-once

. Yeditepe University. (2025). Cse331-operating systems design] [Course Material, Accessed via YULearn]. https://yulearn.yeditepe.edu.tr/course/view.php?id=44356

. Sigma2 Training Portal. (2024). Shell scripts and tips: Using awk in scripts [Accessed: 2025-12-19]. https://training.pages.sigma2.no/tutorials/shell-scripts-and-tips/episodes/09-awk.html

. Gite, V. (2023). Bash scripting: Using awk to process data [Accessed: 2025-12-19]. https://www.cyberciti.biz/faq/bash-scripting-using-awk/

. UNIX Community Contributors. (2014). Awk script for standard deviation / root mean square deviation [Accessed: 2025-12-19]. https://community.unix.com/t/awk-script-for-standard-deviation-root-mean-square-deviation/302659

. Waldspurger, C. A., & Weihl, W. E. (1994). Lottery scheduling: Flexible proportional-share resource management. *OSDI*, *1*, 1–11.